

CST2550 - Software Engineering and Management

Laboratory Exercises

Lab 1: Introduction to Software Engineering and C# .NET

Duration: 2 hours

Prerequisites: Visual Studio 2022 installed

Objectives

- Set up the development environment
- Create first C# application
- Understand basic OOP concepts

Tasks

1. Environment Setup (20 minutes)
 - o Launch Visual Studio 2022
 - o Create a new C# Console Application
 - o Explore the IDE interface
 - o Understand solution explorer
2. Basic C# Programming (40 minutes)

```
// Create a Person class with the following:
public class Person
{
    // Properties
    public string Name { get; set; }
    public int Age { get; set; }
    public string Email { get; set; }

    // Constructor
    // Methods for validation
    // ToString override
}
```

- Implement the constructor
 - Add validation methods
 - Override ToString()
3. Student Management System (60 minutes)
 - o Create a StudentManager class
 - o Implement methods to:

- Add new students
- Display student information
- Search for students
- Calculate average age

Expected Outcomes

- Working C# development environment
- Understanding of basic OOP concepts
- Simple student management system

Extension Activities

- Add more properties to Person class
 - Implement data validation
 - Add exception handling
 - Create unit tests
-

Lab 2: Data Structures - Arrays and Lists

Duration: 2 hours

Prerequisites: Completed Lab 1

Objectives

- Understand array operations
- Implement List<T>
- Compare array and list performance

Tasks

1. Array Operations (40 minutes)

```
public class ArrayOperations
{
    // Implement methods for:
    public void ReverseArray(int[] arr)
    public int FindMaximum(int[] arr)
    public bool SearchElement(int[] arr, int target)
    public void SortArray(int[] arr)
}
```

2. List Implementation (40 minutes)

```
public class StudentList
{
    private List<Student> students;

    // Implement methods for:
    public void AddStudent(Student student)
    public void RemoveStudent(string id)
    public Student FindStudent(string id)
    public List<Student> GetStudentsByAge(int age)
}
```

3. Performance Comparison (40 minutes)

- Create large datasets
- Compare operation speeds
- Document findings

Lab 3: Advanced Data Structures - Graphs and Hash Tables

Duration: 2 hours

Prerequisites: Completed Labs 1-2

Objectives

- Implement a hash table
- Create a graph structure
- Perform graph traversal

Tasks

1. Hash Table Implementation (40 minutes)

```
public class SimpleHashTable<TKey, TValue>
{
    private LinkedList<KeyValuePair<TKey, TValue>>[] buckets;
    private int size;

    // Implement:
    public void Add(TKey key, TValue value)
    public bool Remove(TKey key)
    public TValue Get(TKey key)
    public bool Contains(TKey key)
}
```

2. Graph Implementation (40 minutes)

```
public class Graph<T>
{
    private Dictionary<T, List<T>> adjacencyList;

    // Implement:
    public void AddVertex(T vertex)
    public void AddEdge(T source, T destination)
    public void RemoveVertex(T vertex)
    public void RemoveEdge(T source, T destination)
    public List<T> GetNeighbors(T vertex)
}
```

3. Graph Traversal (40 minutes)

- Implement Depth-First Search (DFS)
- Implement Breadth-First Search (BFS)
- Find shortest path between vertices

Extension Activities

- Implement weighted graphs
- Add cycle detection
- Create visualization of graph structure

Lab 4: Introduction to Database and SQL

Duration: 2 hours

Prerequisites: SQL Server Management Studio installed

Objectives

- Create database schemas
- Write SQL queries
- Understand relationships

Tasks

1. Database Design (30 minutes)

```
-- Create tables for a library management system
CREATE TABLE Books (
    BookID INT PRIMARY KEY IDENTITY(1,1),
    Title VARCHAR(100) NOT NULL,
    Author VARCHAR(100),
    ISBN VARCHAR(13),
    PublishedYear INT
);

CREATE TABLE Members (
    MemberID INT PRIMARY KEY IDENTITY(1,1),
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    Email VARCHAR(100) UNIQUE
);

-- Create lending table with relationships
```

2. Data Manipulation (45 minutes)

```
-- Insert sample data
-- Write queries for:
-- 1. Find all books by specific author
-- 2. List members with borrowed books
-- 3. Calculate overdue books
-- 4. Generate lending statistics
```

3. Complex Queries (45 minutes)

- Write joins
- Create views
- Implement stored procedures
- Use transactions

Extension Activities

- Add more complex relationships

- Implement triggers
 - Create backup procedures
-

Lab 5: Implementing Database in C# Application

Duration: 2 hours

Prerequisites: Completed Lab 4, Entity Framework Core installed

Objectives

- Connect C# to database
- Implement CRUD operations
- Use Entity Framework Core

Tasks

1. Database Connection (30 minutes)

```
public class DatabaseContext : DbContext
{
    public DbSet<Book> Books { get; set; }
    public DbSet<Member> Members { get; set; }
    public DbSet<Lending> Lendings { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        // Configure connection string
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Configure relationships
    }
}
```

2. Repository Implementation (45 minutes)

```
public interface IBookRepository
{
    Task<Book> GetByIdAsync(int id);
    Task<IEnumerable<Book>> GetAllAsync();
    Task AddAsync(Book book);
    Task UpdateAsync(Book book);
    Task DeleteAsync(int id);
}

// Implement repository pattern
```

3. Business Logic Layer (45 minutes)

- o Implement service layer
- o Add validation logic
- o Handle transactions
- o Implement error handling

Extension Activities

- Add caching mechanism
 - Implement unit tests
 - Create API endpoints
-

Lab 6: Version Control Systems - Git and Azure Repos

Duration: 2 hours

Prerequisites: Git installed, Azure DevOps account

Objectives

- Use Git commands
- Manage branches
- Work with Azure Repos

Tasks

1. Git Basics (30 minutes)

```
# Initialize repository
git init

# Create and switch to feature branch
git checkout -b feature/new-feature

# Stage and commit changes
git add .
git commit -m "Implement new feature"

# Push to remote
git remote add origin <url>
git push -u origin feature/new-feature
```

2. Branching and Merging (45 minutes)

- Create feature branches
- Resolve merge conflicts
- Create pull requests
- Review code changes

3. Azure Repos Integration (45 minutes)

- Set up Azure repository
- Configure branch policies
- Implement CI/CD pipeline
- Review build results

Extension Activities

- Create complex branching strategy
- Implement git hooks
- Set up automated testing

Lab 7: Debugging and Testing Software

Duration: 2 hours

Prerequisites: Completed previous labs

Objectives

- Debug C# applications
- Write unit tests
- Perform integration testing

Tasks

1. Debugging Practice (40 minutes)

```
public class OrderProcessor
{
    public decimal CalculateTotal(Order order)
    {
        // Intentionally buggy code for debugging
        decimal total = 0;
        foreach (var item in order.Items)
        {
            total += item.Price * item.Quantity;
            // Add tax calculation
            // Apply discounts
        }
        return total;
    }
}
```

2. Unit Testing (40 minutes)

```
[TestClass]
public class OrderProcessorTests
{
    private OrderProcessor _processor;

    [TestInitialize]
    public void Setup()
    {
        _processor = new OrderProcessor();
    }

    [TestMethod]
    public void CalculateTotal_WithValidOrder_ReturnsCorrectTotal()
    {
        // Arrange
        // Act
        // Assert
    }
}
```

3. Integration Testing (40 minutes)

- Test database operations
- Test API endpoints
- Test user interface

Extension Activities

- Add mock objects
 - Implement test coverage
 - Create automated UI tests
-

Lab 8: Analysis of Algorithms

Duration: 2 hours

Prerequisites: Completed previous labs

Objectives

- Analyze algorithm complexity
- Measure performance
- Optimize code

Tasks

1. Algorithm Implementation (40 minutes)

```
public class SortingAlgorithms
{
    // Implement and analyze:
    public void BubbleSort(int[] arr)
    public void QuickSort(int[] arr)
    public void MergeSort(int[] arr)

    // Compare performance
    public void CompareAlgorithms(int[] data)
}
```

2. Performance Testing (40 minutes)

- Create test data sets
- Measure execution time
- Compare memory usage
- Document results

3. Optimization (40 minutes)

- Identify bottlenecks
- Improve algorithms
- Validate improvements

Extension Activities

- Implement parallel algorithms
- Create visualization of sorting
- Add more complex algorithms

Lab 9: Software Development Life Cycle

Duration: 2 hours

Prerequisites: Azure DevOps access

Objectives

- Practice SDLC phases
- Create project documentation
- Manage requirements

Tasks

1. Project Setup (40 minutes)
 - Create project charter
 - Define requirements
 - Design system architecture
 - Plan iterations
2. Implementation Planning (40 minutes)
 - Break down tasks
 - Estimate effort
 - Assign resources
 - Create timeline
3. Documentation (40 minutes)
 - Write technical specifications
 - Create user stories
 - Design test cases
 - Plan deployment

Extension Activities

- Create risk management plan
 - Develop quality metrics
 - Design monitoring strategy
-

Lab 10: Agile Methodologies and MS Azure DevOps

Duration: 2 hours

Prerequisites: Azure DevOps access

Objectives

- Use Azure Boards
- Manage sprints
- Track progress

Tasks

1. Sprint Planning (40 minutes)
 - Create product backlog
 - Define user stories
 - Estimate story points
 - Plan sprint capacity
2. Azure Boards Setup (40 minutes)

```
// Example user story structure
public class UserStory
{
    public string Title { get; set; }
    public string AsA { get; set; }
    public string IWant { get; set; }
    public string SoThat { get; set; }
    public List<string> AcceptanceCriteria { get; set; }
}
```

3. Progress Tracking (40 minutes)
 - Update work items
 - Track burndown
 - Conduct daily standups
 - Review sprint progress

Extension Activities

- Configure dashboards
- Create custom reports
- Implement retrospectives

Lab 11: The Principles of Parallel Processing and Multi-Threading in C#

Duration: 2 hours

Prerequisites: Visual Studio 2022, basic understanding of C# programming

Objectives

- Understand threading concepts
- Implement parallel processing
- Handle thread synchronization
- Use async/await patterns

Tasks

1. Basic Threading Concepts (40 minutes)

```
public class ThreadingBasics
{
    // Task 1: Create and manage threads
    public void DemonstrateBasicThreading()
    {
        // Create a new thread
        Thread thread = new Thread(() =>
        {
            for (int i = 0; i < 5; i++)
            {
                Console.WriteLine($"Thread {Thread.CurrentThread.ManagedThreadId}:
Count {i}");
                Thread.Sleep(1000);
            }
        });

        // Start the thread
        thread.Start();

        // Wait for thread completion
        thread.Join();
    }

    // Task 2: Implement thread pool
    public void ThreadPoolExample()
    {
        for (int i = 0; i < 5; i++)
        {
            ThreadPool.QueueUserWorkItem(state =>
            {
                Console.WriteLine($"Task {state} executing on thread
{Thread.CurrentThread.ManagedThreadId}");
                Thread.Sleep(1000);
            }, i);
        }
    }
}
```

2. Parallel Processing (40 minutes)

```

public class ParallelProcessing
{
    // Task 1: Implement parallel loops
    public void ProcessDataInParallel(List<int> numbers)
    {
        Parallel.ForEach(numbers, number =>
        {
            // Simulate processing
            Thread.Sleep(100);
            Console.WriteLine($"Processing {number} on thread
{Thread.CurrentThread.ManagedThreadId}");
        });
    }

    // Task 2: Implement parallel LINQ
    public void ParallelLINQExample(List<int> numbers)
    {
        var evenNumbers = numbers.AsParallel()
            .Where(n => n % 2 == 0)
            .OrderBy(n => n)
            .ToList();

        Console.WriteLine("Even numbers: " + string.Join(", ", evenNumbers));
    }

    // Task 3: Task Parallel Library usage
    public async Task ProcessTasksAsync()
    {
        var tasks = new List<Task<int>>();

        for (int i = 0; i < 5; i++)
        {
            int taskNum = i;
            tasks.Add(Task.Run(() => ProcessTask(taskNum)));
        }

        var results = await Task.WhenAll(tasks);
        Console.WriteLine("All tasks completed: " + string.Join(", ", results));
    }

    private async Task<int> ProcessTask(int taskNum)
    {
        await Task.Delay(1000); // Simulate work
        return taskNum * 2;
    }
}

```

3. Thread Synchronization (40 minutes)

```

public class ThreadSynchronization
{
    private readonly object _lockObject = new object();
    private int _sharedResource = 0;
}

```

```
// Task 1: Implement lock statement
public void DemonstrateLocking()
{
    var tasks = new List<Task>();

    for (int i = 0; i < 100; i++)
    {
        tasks.Add(Task.Run(() =>
        {
            lock (_lockObject)
            {
                _sharedResource++;
                Console.WriteLine($"Counter: {_sharedResource}");
            }
        }));
    }

    Task.WaitAll(tasks.ToArray());
}

// Task 2: Implement Monitor
public void DemonstrateMonitor()
{
    bool lockTaken = false;
    try
    {
        Monitor.Enter(_lockObject, ref lockTaken);
        // Critical section
        _sharedResource++;
    }
    finally
    {
        if (lockTaken)
        {
            Monitor.Exit(_lockObject);
        }
    }
}

// Task 3: Implement producer-consumer pattern
public class ProducerConsumer
{
    private readonly Queue<int> _queue = new Queue<int>();
    private readonly SemaphoreSlim _semaphore = new SemaphoreSlim(0, 10);
    private readonly object _lockObject = new object();

    public async Task ProduceAsync()
    {
        for (int i = 0; i < 10; i++)
        {
            lock (_lockObject)
            {
                _queue.Enqueue(i);
                Console.WriteLine($"Produced: {i}");
            }
        }
    }
}
```

```

        }
        _semaphore.Release();
        await Task.Delay(100);
    }
}

public async Task ConsumeAsync()
{
    while (true)
    {
        await _semaphore.WaitAsync();
        int item;
        lock (_lockObject)
        {
            if (_queue.Count == 0) break;
            item = _queue.Dequeue();
        }
        Console.WriteLine($"Consumed: {item}");
    }
}
}
}
}

```

Extension Activities

1. Advanced Threading
 - Implement custom thread pool
 - Create thread-safe collections
 - Handle thread cancellation
2. Performance Analysis
 - Compare sequential vs parallel execution
 - Measure thread overhead
 - Analyze thread pool efficiency
3. Real-world Scenarios
 - Implement parallel web scraper
 - Create multi-threaded file processor
 - Develop concurrent database operations

Scaffolding for Weaker Students

4. Basic Thread Creation

```

// Template for creating a simple thread
public void CreateBasicThread()
{
    // TODO: Create a thread that prints numbers 1-5
}

```

```
} // Hint: Use new Thread(() => { /* Your code here */ })
```

2. Simple Parallel Loop

```
// Template for parallel processing
public void SimpleParallelLoop()
{
    var numbers = Enumerable.Range(1, 10).ToList();
    // TODO: Process numbers in parallel
    // Hint: Use Parallel.ForEach(numbers, number => { /* Your code */ })
}
```

3. Basic Async/Await

```
// Template for async operations
public async Task BasicAsyncOperation()
{
    // TODO: Create and await a simple task
    // Hint: Use await Task.Run(() => { /* Your code */ })
}
```

Assessment Criteria

- Understanding of threading concepts (25%)
- Implementation of parallel processing (25%)
- Thread synchronization handling (25%)
- Code quality and best practices (25%)

Additional Resources

- Microsoft Documentation on Threading
 - C# Threading Best Practices
 - Parallel Programming Patterns
 - Threading Code Samples Repository
-

Lab 12: Software Design Patterns and Principles

Duration: 2 hours

Prerequisites: Completed previous labs

Objectives

- Implement design patterns
- Apply SOLID principles
- Create flexible designs

Tasks

1. Pattern Implementation (40 minutes)

```
// Implement patterns:  
public class SingletonLogger  
public abstract class PaymentFactory  
public class OrderObserver  
public interface IPaymentStrategy
```

2. SOLID Principles (40 minutes)
 - Refactor existing code
 - Apply principles
 - Improve design
3. Design Review (40 minutes)
 - Review implementations
 - Suggest improvements
 - Document patterns

Extension Activities

- Combine multiple patterns
- Create pattern catalog
- Design new patterns

Final Group Coursework Project

Students should combine knowledge from all labs to create a complete software solution incorporating:

- Clean architecture
- Design patterns
- Testing strategy
- Performance optimization
- Documentation

The projects ideas will be allocated to each group by The Syndicate.